

Floyd–Warshall algorithm

From Wikipedia, the free encyclopedia

In computer science, the **Floyd–Warshall algorithm** is an algorithm for finding shortest paths in a weighted graph with positive or negative edge weights (but with no negative cycles).^{[1][2]} A single execution of the algorithm will find the lengths (summed weights) of the shortest paths between *all* pairs of vertices, though it does not return details of the paths themselves. Versions of the algorithm can also be used for finding the transitive closure of a relation *R*, or (in connection with the Schulze voting system) widest paths between all pairs of vertices in a weighted graph.

Contents

- 1 History and naming
- 2 Algorithm
- 3 Example
- 4 Behavior with negative cycles
- 5 Path reconstruction
- 6 Analysis
- 7 Applications and generalizations
- 8 Implementations
- 9 Comparison with other shortest path algorithms
- 10 References
- 11 External links

Floyd–Warshall algorithm

Class	All-pairs shortest path problem (for weighted graphs)
Data structure	Graph
Worst case performance	$O(V ^3)$
Best case performance	$\Omega(V ^3)$
Worst case space complexity	$\Theta(V ^2)$

History and naming

The Floyd–Warshall algorithm is an example of dynamic programming, and was published in its currently recognized form by Robert Floyd in 1962.^[3] However, it is essentially the same as algorithms previously published by Bernard Roy in 1959^[4] and also by Stephen Warshall in 1962^[5] for finding the transitive closure of a graph,^[6] and is closely related to Kleene's algorithm (published in 1956) for converting a deterministic finite automaton into a regular expression.^[7] The modern formulation of the algorithm as three nested for-loops was first described by Peter Ingerman, also in 1962.^[8]

The algorithm is also known as **Floyd's algorithm**, the **Roy–Warshall algorithm**, the **Roy–Floyd algorithm**, or the **WFI algorithm**.

Algorithm

The Floyd–Warshall algorithm compares all possible paths through the graph between each pair of vertices. It is able to do this with $\Theta(|V|^3)$ comparisons in a graph. This is remarkable considering that there may be up to $\Omega(|V|^2)$ edges in the graph, and every combination of edges is tested. It does so by incrementally improving an estimate on the shortest path between two vertices, until the estimate is optimal.

Consider a graph G with vertices V numbered 1 through N . Further consider a function $\text{shortestPath}(i, j, k)$ that returns the shortest possible path from i to j using vertices only from the set $\{1, 2, \dots, k\}$ as intermediate points along the way. Now, given this function, our goal is to find the shortest path from each i to each j using only vertices 1 to $k + 1$.

For each of these pairs of vertices, the true shortest path could be either

(1) a path that only uses vertices in the set $\{1, \dots, k\}$

or

(2) a path that goes from i to $k + 1$ and then from $k + 1$ to j .

We know that the best path from i to j that only uses vertices 1 through k is defined by $\text{shortestPath}(i, j, k)$, and it is clear that if there were a better path from i to $k + 1$ to j , then the length of this path would be the concatenation of the shortest path from i to $k + 1$ (using vertices in $\{1, \dots, k\}$) and the shortest path from $\{k + 1\}$ to j (also using vertices in $\{1, \dots, k\}$).

If $w(i, j)$ is the weight of the edge between vertices i and j , we can define $\text{shortestPath}(i, j, k + 1)$ in terms of the following recursive formula: the base case is

$$\text{shortestPath}(i, j, 0) = w(i, j)$$

and the recursive case is

$$\text{shortestPath}(i, j, k+1) = \min(\text{shortestPath}(i, j, k), \text{shortestPath}(i, k+1, k) + \text{shortestPath}(k+1, j, k))$$

This formula is the heart of the Floyd–Warshall algorithm. The algorithm works by first computing $\text{shortestPath}(i, j, k)$ for all (i, j) pairs for $k = 1$, then $k = 2$, etc. This process continues until $k = N$, and we have found the shortest path for all (i, j) pairs using any intermediate vertices. Pseudocode for this basic version follows:

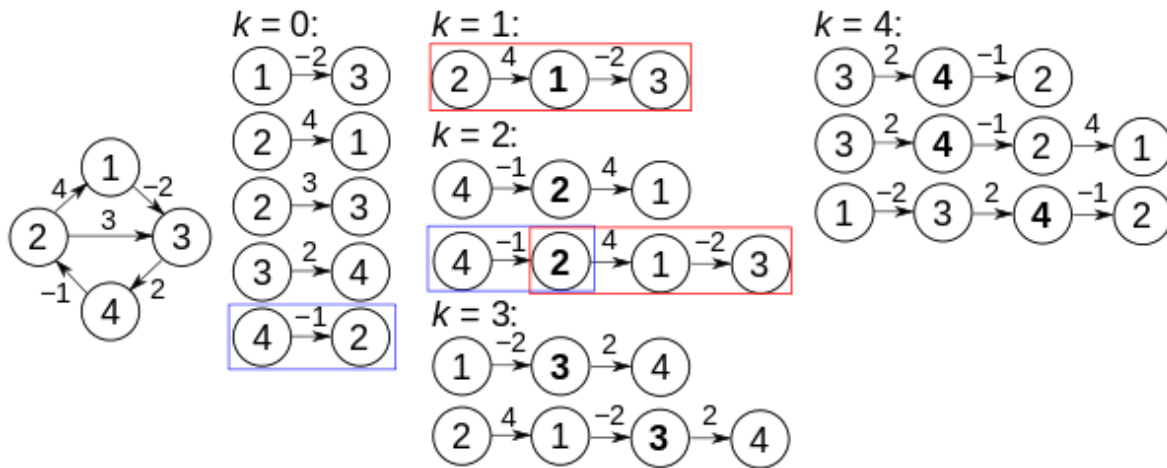
```

1 let dist be a  $|V| \times |V|$  array of minimum distances initialized to  $\infty$  (infinity)
2 for each vertex  $v$ 
3    $\text{dist}[v][v] \leftarrow 0$ 
4 for each edge  $(u, v)$ 
5    $\text{dist}[u][v] \leftarrow w(u, v)$  // the weight of the edge  $(u, v)$ 
6 for  $k$  from 1 to  $|V|$ 
7   for  $i$  from 1 to  $|V|$ 
8     for  $j$  from 1 to  $|V|$ 
9       if  $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ 
10         $\text{dist}[i][j] \leftarrow \text{dist}[i][k] + \text{dist}[k][j]$ 
11      end if

```

Example

The algorithm above is executed on the graph on the left below:



Prior to the first iteration of the outer loop, labeled $k=0$ above, the only known paths correspond to the single edges in the graph. At $k=1$, paths that go through the vertex 1 are found: in particular, the path $2 \rightarrow 1 \rightarrow 3$ is found, replacing the path $2 \rightarrow 3$ which has fewer edges but is longer. At $k=2$, paths going through the vertices $\{1,2\}$ are found. The red and blue boxes show how the path $4 \rightarrow 2 \rightarrow 1 \rightarrow 3$ is assembled from the two known paths $4 \rightarrow 2$ and $2 \rightarrow 1 \rightarrow 3$ encountered in previous iterations, with 2 in the intersection. The path $4 \rightarrow 2 \rightarrow 3$ is not considered, because $2 \rightarrow 1 \rightarrow 3$ is the shortest path encountered so far from 2 to 3. At $k=3$, paths going through the vertices $\{1,2,3\}$ are found. Finally, at $k=4$, all shortest paths are found.

Behavior with negative cycles

A negative cycle is a cycle whose edges sum to a negative value. There is no shortest path between any pair of vertices i, j which form part of a negative cycle, because path-lengths from i to j can be arbitrarily small (negative). For numerically meaningful output, the Floyd–Warshall algorithm assumes that there are no negative cycles. Nevertheless, if there are negative cycles, the Floyd–Warshall algorithm can be used to detect them. The intuition is as follows:

- The Floyd–Warshall algorithm iteratively revises path lengths between all pairs of vertices (i, j) , including where $i = j$;
- Initially, the length of the path (i, i) is zero;
- A path $\{(i, k), (k, i)\}$ can only improve upon this if it has length less than zero, i.e. denotes a negative cycle;
- Thus, after the algorithm, (i, i) will be negative if there exists a negative-length path from i back to i .

Hence, to detect negative cycles using the Floyd–Warshall algorithm, one can inspect the diagonal of the path matrix, and the presence of a negative number indicates that the graph contains at least one negative cycle.^[9] To avoid numerical problems one should check for negative numbers on the diagonal of the path matrix within the inner for loop of the algorithm.^[10] Obviously, in an undirected graph a negative edge creates a negative cycle (i.e., a closed walk) involving its incident vertices. Considering all edges of the above example graph as undirected, e.g. the vertex sequence $4 - 2 - 4$ is a cycle with weight sum -2 .

Path reconstruction

The Floyd–Warshall algorithm typically only provides the lengths of the paths between all pairs of vertices. With simple modifications, it is possible to create a method to reconstruct the actual path between any two endpoint vertices. While one may be inclined to store the actual path from each vertex to each other vertex, this is not necessary, and in fact, is very costly in terms of memory. Instead, the Shortest-path tree can be calculated for each node in $\Theta(|E|)$ time using $\Theta(|V|)$ memory to store each tree which allows us to efficiently reconstruct a path from any two connected vertices.

```

let dist be a  $|V| \times |V|$  array of minimum distances initialized to  $\infty$  (infinity)
let next be a  $|V| \times |V|$  array of vertex indices initialized to null

procedure FloydWarshallWithPathReconstruction ()

```

```

for each edge (u,v)
    dist[u][v] ← w(u,v) // the weight of the edge (u,v)
    next[u][v] ← v
for k from 1 to |V| // standard Floyd-Warshall implementation
    for i from 1 to |V|
        for j from 1 to |V|
            if dist[i][k] + dist[k][j] < dist[i][j] then
                dist[i][j] ← dist[i][k] + dist[k][j]
                next[i][j] ← next[i][k]

procedure Path(u, v)
    if next[u][v] = null then
        return []
    path = [u]
    while u ≠ v
        u ← next[u][v]
        path.append(u)
    return path

```

Analysis

Let n be $|V|$, the number of vertices. To find all n^2 of $\text{shortestPath}(i,j,k)$ (for all i and j) from those of $\text{shortestPath}(i,j,k-1)$ requires $2n^2$ operations. Since we begin with $\text{shortestPath}(i,j,0) = \text{edgeCost}(i,j)$ and compute the sequence of n matrices $\text{shortestPath}(i,j,1)$, $\text{shortestPath}(i,j,2)$, ..., $\text{shortestPath}(i,j,n)$, the total number of operations used is $n \cdot 2n^2 = 2n^3$. Therefore, the complexity of the algorithm is $\Theta(n^3)$.

Applications and generalizations

The Floyd–Warshall algorithm can be used to solve the following problems, among others:

- Shortest paths in directed graphs (Floyd's algorithm).
- Transitive closure of directed graphs (Warshall's algorithm). In Warshall's original formulation of the algorithm, the graph is unweighted and represented by a Boolean adjacency matrix. Then the addition operation is replaced by logical conjunction (AND) and the minimum operation by logical disjunction (OR).
- Finding a regular expression denoting the regular language accepted by a finite automaton (Kleene's algorithm, a closely related generalization of the Floyd–Warshall algorithm)^[11]
- Inversion of real matrices (Gauss–Jordan algorithm)^[12]
- Optimal routing. In this application one is interested in finding the path with the maximum flow between two vertices. This means that, rather than taking minima as in the pseudocode above, one instead takes maxima. The edge weights represent fixed constraints on flow. Path weights represent bottlenecks; so the addition operation above is replaced by the minimum operation.
- Fast computation of Pathfinder networks.
- Widest paths/Maximum bandwidth paths
- Computing canonical form of difference bound matrices (DBMs)

Implementations

Implementations are available for many programming languages.

- For C++, in the `boost::graph` (<http://www.boost.org/libs/graph/doc/>) library
- For C#, at QuickGraph (<http://www.codeplex.com/quickgraph>)
- For Java, in the Apache Commons Graph (<http://commons.apache.org/sandbox/commons-graph/>) library
- For MATLAB, in the Matlab_bgl (<http://www.mathworks.com/matlabcentral/fileexchange/10922>) package
- For Perl, in the Graph (<https://metacpan.org/module/Graph>) module
- For Python, in the SciPy library (module `scipy.sparse.csgraph` (http://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.floyd_warshall.html#scipy.sparse.csgraph.floyd_warshall)) or NetworkX library
- For R, in package `e1071` (<http://cran.r-project.org/web/packages/e1071/index.html>)

Comparison with other shortest path algorithms

The Floyd–Warshall algorithm is a good choice for computing paths between all pairs of vertices in dense graphs, in which most or all pairs of vertices are connected by edges. For sparse graphs with non-negative edge weights, a better choice is to use Dijkstra's algorithm from each possible starting vertex, since the running time of repeated Dijkstra ($O(|V| \cdot |E| \log |V|)$ using binary heaps) is better than the $O(|V|^3)$ running time of the Floyd–Warshall algorithm when $|E|$ is significantly smaller than $|V|^2$. For sparse graphs with negative edges but no negative cycles, Johnson's algorithm can be used, with the same asymptotic running time as the repeated Dijkstra approach.

There are also known algorithms using fast matrix multiplication to speed up all-pairs shortest path computation in dense graphs, but these typically make extra assumptions on the edge weights (such as requiring them to be small integers).^{[13][14]} In addition, because of the high constant factors in their running time, they would only provide a speedup over the Floyd–Warshall algorithm for very large graphs.

References

1. Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L. (1990). *Introduction to Algorithms* (1st ed.). MIT Press and McGraw-Hill. ISBN 0-262-03141-8. See in particular Section 26.2, "The Floyd–Warshall algorithm", pp. 558–565 and Section 26.4, "A general framework for solving path problems in directed graphs", pp. 570–576.
2. Kenneth H. Rosen (2003). *Discrete Mathematics and Its Applications, 5th Edition*. Addison Wesley. ISBN 0-07-119881-4.
3. Floyd, Robert W. (June 1962). "Algorithm 97: Shortest Path". *Communications of the ACM* **5** (6): 345. doi:10.1145/367766.368168.
4. Roy, Bernard (1959). "Transitivité et connexité.". *C. R. Acad. Sci. Paris* **249**: 216–218.
5. Warshall, Stephen (January 1962). "A theorem on Boolean matrices". *Journal of the ACM* **9** (1): 11–12. doi:10.1145/321105.321107.
6. Weisstein, Eric W., "Floyd–Warshall Algorithm" (<http://mathworld.wolfram.com/Floyd-WarshallAlgorithm.html>), *MathWorld*.
7. Kleene, S. C. (1956). "Representation of events in nerve nets and finite automata". In C. E. Shannon and J. McCarthy. *Automata Studies*. Princeton University Press. pp. 3–42.
8. Ingberman, Peter Z. (November 1962). "Algorithm 141: Path Matrix". *Communications of the ACM* **5** (11): 556. doi:10.1145/368996.369016.
9. Hochbaum, Dorit (2014). "Section 8.9: Floyd–Warshall algorithm for all pairs shortest paths" (PDF). *Lecture Notes for IEOR 266: Graph Algorithms and Network Flows*. Department of Industrial Engineering and Operations Research, University of California, Berkeley.
10. Stefan Hougardy (April 2010). "The Floyd–Warshall algorithm on graphs with negative cycles". *Information Processing Letters* **110** (8-9): 279–281. doi:10.1016/j.ipl.2010.02.001.
11. Gross, Jonathan L.; Yellen, Jay (2003), *Handbook of Graph Theory*, Discrete Mathematics and Its Applications, CRC Press, p. 65, ISBN 9780203490204.
12. Penaloza, Rafael. "Algebraic Structures for Transitive Closure".
13. Zwick, Uri (May 2002), "All pairs shortest paths using bridging sets and rectangular matrix multiplication", *Journal of the ACM* **49** (3): 289–317, doi:10.1145/567112.567114.
14. Chan, Timothy M. (January 2010), "More algorithms for all-pairs shortest paths in weighted graphs", *SIAM Journal on Computing* **39** (5): 2075–2089, doi:10.1137/08071990x.

External links

- Interactive animation of the Floyd–Warshall algorithm ([http://www.pms.informatik.uni-](http://www.pms.informatik.uni-muenchen.de/lehre/compgeometry/Gosper/shortest_path/shortest_path.html#visualization)

[muenchen.de/lehre/compgeometry/Gosper/shortest_path/shortest_path.html#visualization](http://www.pms.informatik.uni-muenchen.de/lehre/compgeometry/Gosper/shortest_path/shortest_path.html#visualization))

- Visualization of Floyd's algorithm (http://students.ceid.upatras.gr/~papagel/english/java_docs/allmin.htm)



Wikimedia Commons has media related to ***Floyd–Warshall algorithm***.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Floyd–Warshall_algorithm&oldid=700575112"

Categories: [Graph algorithms](#) | [Routing algorithms](#) | [Polynomial-time problems](#) | [Dynamic programming](#)

- This page was last modified on 19 January 2016, at 09:36.
- Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.